

# Chatroom distribuée SOAP + REST

**Architecture polyglotte avec frontend hexagonal**

Mouhamed Lamine Faye Zeitune

École Supérieure Polytechnique — DGI

Cours Web Services — DIC3 — Mai 2026

# Plan

1. Sujet et fonctionnalités
2. Conception et architecture
3. Modèle de données et intégrité
4. Communication SOAP
5. Communication REST
6. Bilan

# Sujet

Le cas pratique du cours : une chatroom en SOAP et en REST.

# Fonctionnalités

S'inscrire

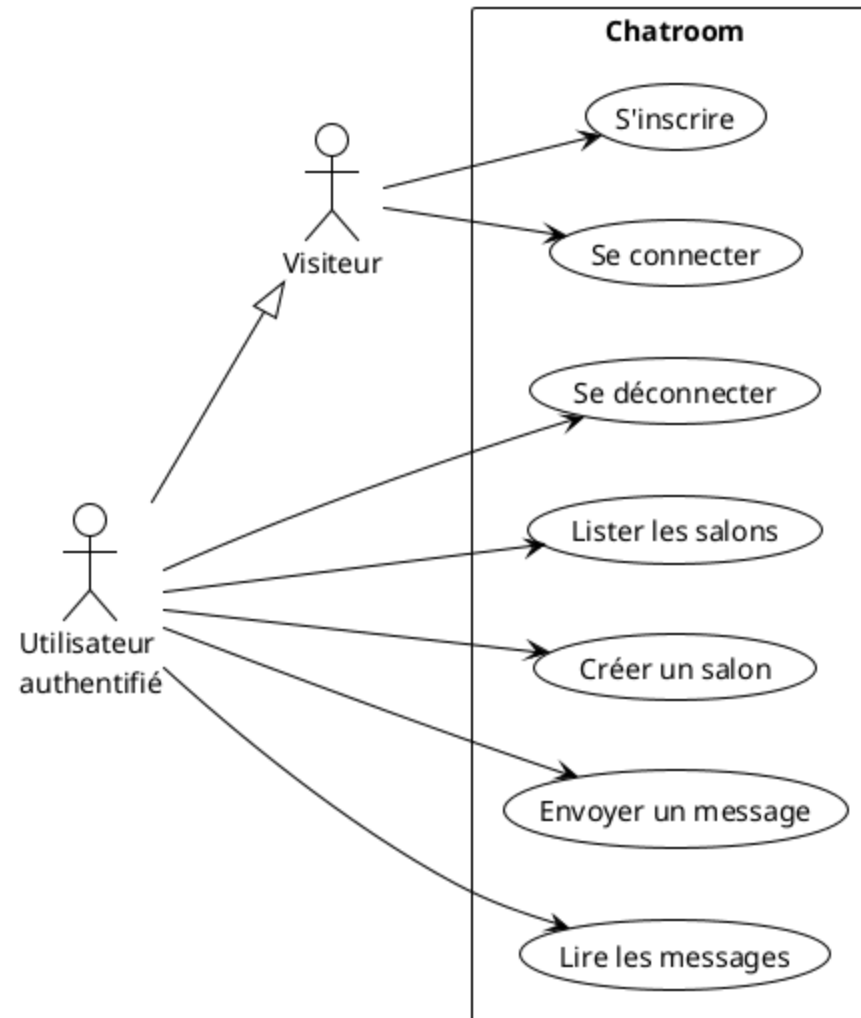
Se connecter

Créer ou rejoindre un salon

Envoyer un message

Voir les messages des autres en temps réel

Se déconnecter



# Architecture

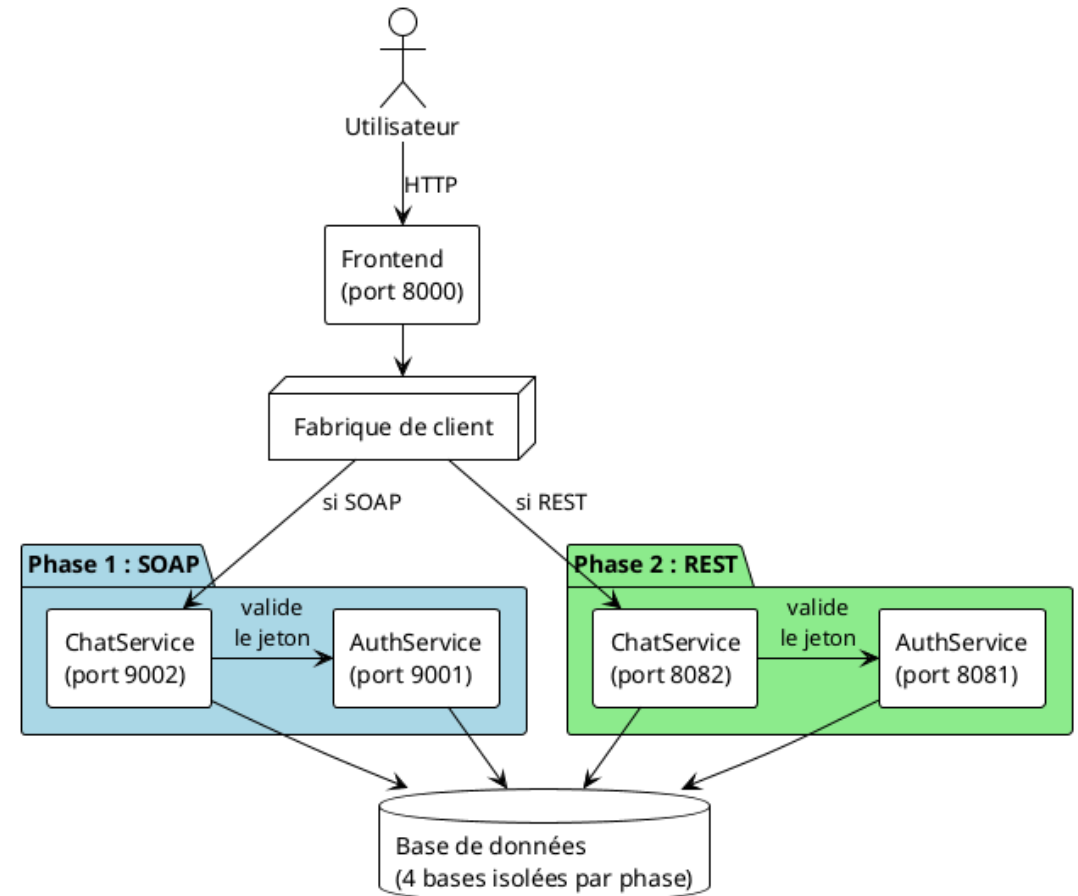
Un même frontend pour deux protocoles

Deux jeux de services backend interchangeable

Validation du jeton à chaque opération

Bases de données isolées par phase

Bascule par variable d'environnement

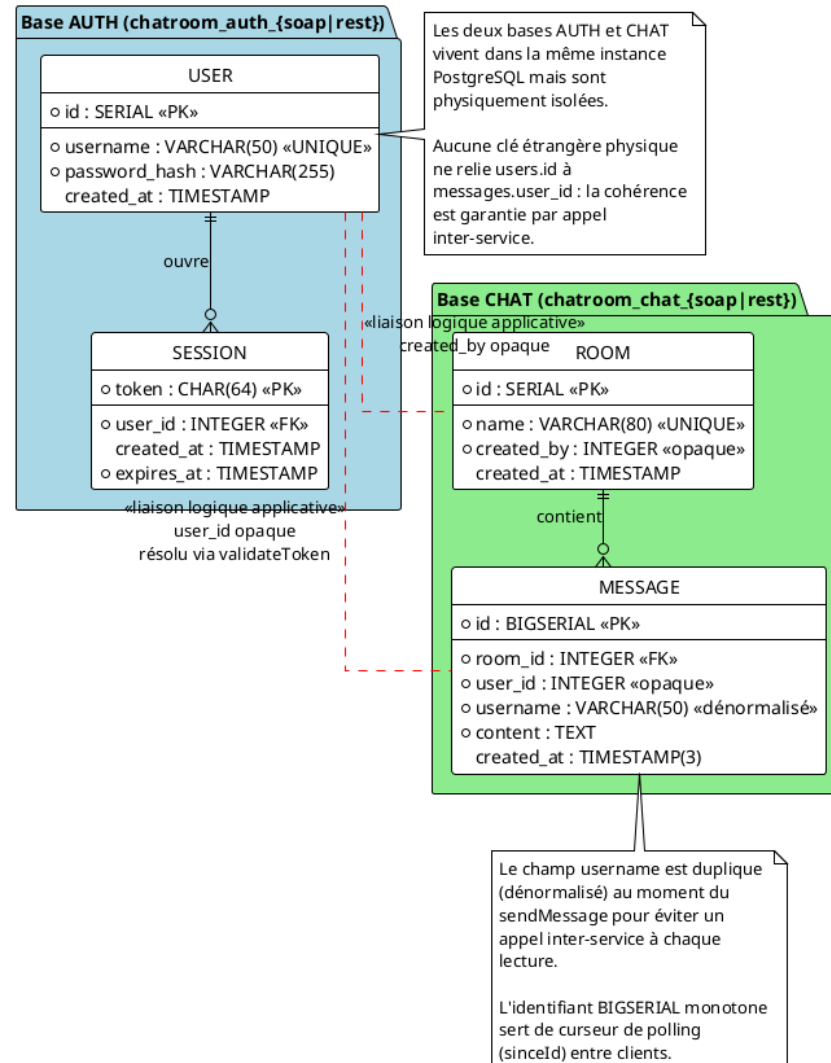


# Modèle conceptuel de données

## Comment garantir l'intégrité sans clé étrangère ?

Les deux bases d'un même backend sont **physiquement isolées**. PostgreSQL n'autorise pas de clé étrangère inter-base.

### Modèle Conceptuel de Données (MCD) — backends isolés

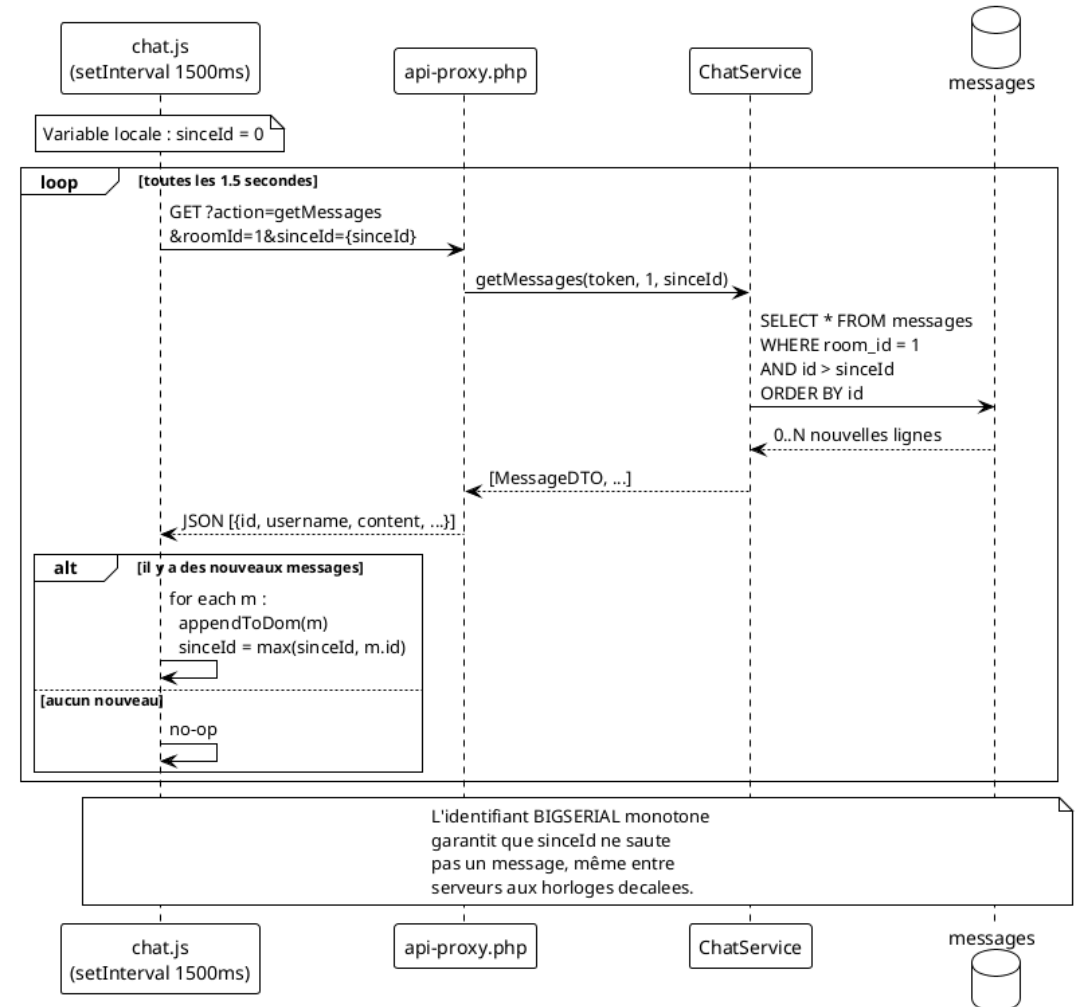


# Polling côté navigateur

## Comment livrer les nouveaux messages en temps réel ?

Le navigateur interroge le serveur toutes les **1,5 secondes** et utilise un curseur `sinceId` monotone pour ne recevoir que les messages plus récents.

Séquence du polling côté navigateur (toutes les 1.5 secondes)



# Stack technique par bloc

| Bloc        | Phase SOAP              | Phase REST                 |
|-------------|-------------------------|----------------------------|
| Frontend    | PHP 8.2 (SoapClient)    | PHP 8.2 (cURL)             |
| AuthService | Java 21 + JAX-WS Metro  | Java 21 + Jersey + Grizzly |
| ChatService | Node.js 20 + node-soap  | Python 3.12 + Flask        |
| BD          | PostgreSQL 16 (2 bases) | PostgreSQL 16 (2 bases)    |
| Sécurité    | bcrypt (cost 10)        | bcrypt (cost 10)           |

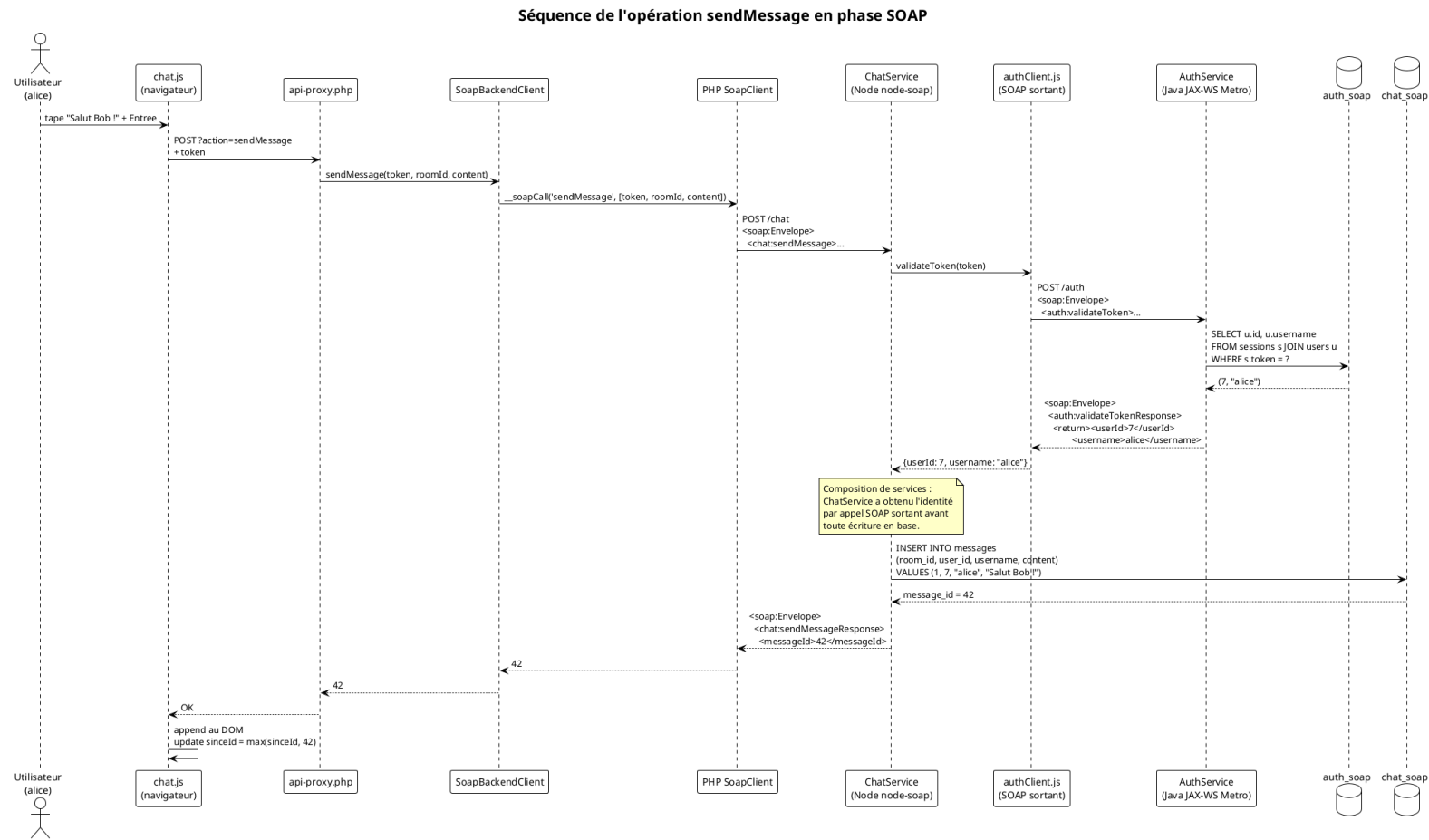
# SOAP : le contrat WSDL

Un WSDL décrit publiquement un service SOAP en **cinq sections imbriquées** :

| Section                       | Rôle                                                          |
|-------------------------------|---------------------------------------------------------------|
| <code>&lt;types&gt;</code>    | dictionnaire XSD des structures (DTOs)                        |
| <code>&lt;message&gt;</code>  | regroupement logique des entrées et sorties                   |
| <code>&lt;portType&gt;</code> | interface abstraite : opérations + entrées + sorties + fautes |
| <code>&lt;binding&gt;</code>  | protocole concret : SOAP sur HTTP, document/literal           |
| <code>&lt;service&gt;</code>  | adresse réelle de l'endpoint (URL)                            |

Un client SOAP **génère ses stubs automatiquement** à partir du WSDL.

# SOAP : envoi d'un message



# SoapUI en action

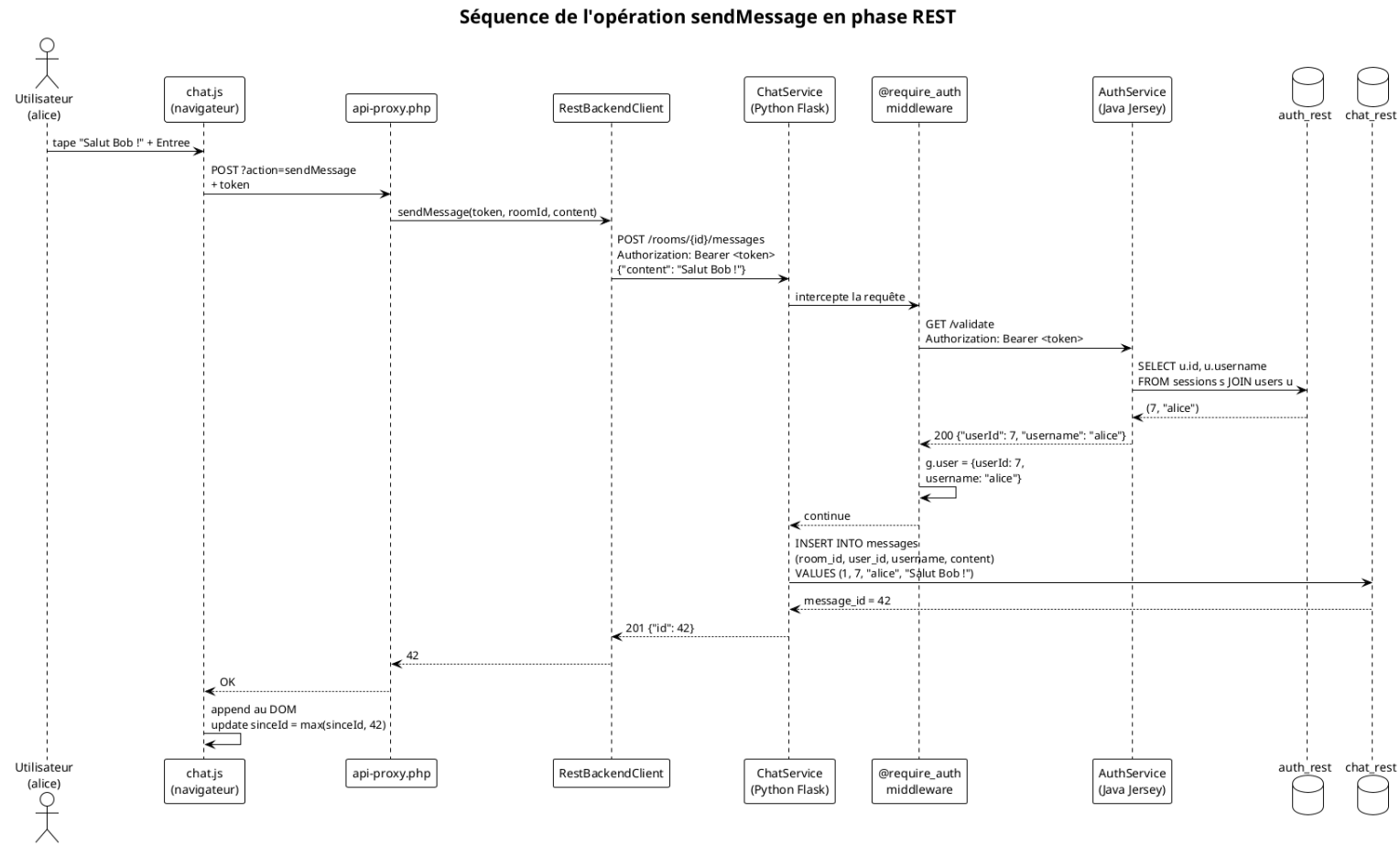
Projet SoapUI : **14 TestCases**, 6 Property Transfers, 23 assertions.  
Test du flux complet avec validation inter-service.

# REST : principes

- **Ressources** identifiées par des URI ( `/rooms` , `/rooms/{id}/messages` )
- **Verbes HTTP** : `POST` créer, `GET` lire, `DELETE` supprimer
- **Codes de statut** : 200, 201, 401, 404, 409
- **Authentication** : en-tête `Authorization: Bearer <token>`
- **Négociation de contenu** : un même endpoint sert JSON ou XML selon `Accept`

```
GET /rooms HTTP/1.1  
Authorization: Bearer abc...  
Accept: application/xml
```

# REST : envoi d'un message



# Postman en action

Collection Postman : **28 requêtes** organisées par service.

Chaque opération existe en **JSON et en XML**, exécutables en boucle via le Collection Runner.

# Bilan

- **Composition de services** observable : chaque opération métier traverse les deux services et illustre la délégation de l'authentification.
- **Polyglossie tenue** : Java, Node.js et Python interagissent sans friction grâce aux contrats standardisés.
- **Bascule SOAP**  
  
**REST** par simple variable d'environnement, sans modification du code applicatif, grâce au pattern **Port et Adaptateurs**.

*Merci de votre attention.*